



SOURCE MAPPING DOCUMENT (SMD)

Document 05 of 09 · B2B Data Engineering Platform



Explains how source data is mapped and transformed into each target layer. The column-level mapping lives in the SMX workbook.

Document ID	B2B-DEP-05
Version	1.2
Status	Final
Date	05/10/2026
Prepared by	Syed Kazim Raza
Reviewed by	[Team Lead]

05/10/2026

DATAROPES.AI

Data Engineering Internship · B2B Data Engineering Platform

Document Control

Revision History

Version	Date	Author	Description of Change
0.1	-	Syed Kazim Raza	Template created
1.0	29/09/2026	Syed Kazim Raza	Content populated from the transform/load scripts; aligned with SMX v0.2 (30 map sets, 350 column mappings)
1.1	30/09/2026	Syed Kazim Raza	Rejects saved in rejected_records (R33); staging emptied after each run (R34); rebuilt data
1.2	05/10/2026	Syed Kazim Raza	Formatting to documentation standard: table column widths, numbered figure and table captions with lists, figure descriptions, complete acronym list

Approvals

Role	Name	Signature	Date
Prepared by	Syed Kazim Raza		
Reviewed by	[Team Lead]		
Approved by	[Manager]		

Related Documents

Ref	Document	Location
R1	Business Requirements Document (BRD)	Provided by DataRopes
R2	Project Document (technical reference)	docs/deliverables/project_document.md
R3	Project Plan	docs/deliverables/project_plan.md
R4	README / Deployment Guide	README.md

Contents

Document Control 2

Contents 3

List of Figures..... 4

List of Tables..... 5

1. Introduction..... 6

 1.1 Purpose..... 6

 1.2 Scope 6

 1.3 Intended Audience 6

 1.4 Definitions & Acronyms..... 6

2. Mapping Scope & Layers 7

3. Mapping Conventions..... 7

4. Object-Level Mapping Summary 8

5. Key Transformation Rules..... 9

6. Deduplication & Latest-Record Logic 9

7. Cleansing & Standardisation..... 10

8. Key Handling..... 10

9. Rejects & Error Handling 11

10. Reference to SMX Workbook 12

11. Assumptions, Constraints & Open Issues 12

List of Figures

Figure 1: Rejected rows by table and reason (intermediate.rejected_records) 12

List of Tables

Table 1: Intended Audience..... 6

Table 2: Definitions & Acronyms 6

Table 3: Mapping Scope & Layers 7

Table 4: Mapping Conventions..... 7

Table 5: Object-Level Mapping Summary 8

Table 6: Key Transformation Rules..... 9

Table 7: Assumptions, Constraints & Open Issues 12

1. Introduction

1.1 Purpose

This document describes the approach, conventions and key rules behind the source-to-target mapping.

1.2 Scope

In scope:

- Mappings from source → staging → intermediate → warehouse → marts
- Transformation conventions, deduplication, cleansing, key handling, rejects

Out of scope:

- Interface mechanics (see ISD)
- Physical table design (see PDM)

1.3 Intended Audience

Table 1: Intended Audience

Audience	How they use this document
Data Engineers	Implement and maintain transformations
QA	Derive mapping test cases
Team Lead	Review mapping logic

1.4 Definitions & Acronyms

Table 2: Definitions & Acronyms

Term	Definition
B2B	Business-to-Business: companies selling to other companies.
BRD	Business Requirements Document: here the Project Requirements Document provided by DataRopes.
Business key	Identifier coming from the source (e.g. company_id, order_id), 32-char hex UUID.
CDC	Change Data Capture: extracting only the rows that changed since the last run (here: watermark on updated_at).
CSV	Comma-Separated Values: the text-file format of the marketing leads and web logs feeds.
CTE	Common Table Expression: a named sub-query (WITH ...) inside a SQL statement.
DEP	Data Engineering Platform: prefix of the document IDs (B2B-DEP-01 ...).
HTTP	Hypertext Transfer Protocol: the web protocol recorded in the web logs (GET, POST ...).
ID	Identifier.
ISD	Interface Specification Document (deliverable 01).
KPI	Key Performance Indicator: a business measure shown on the dashboard (KPI-01 ... KPI-15).
Map Set	One source object feeding one target object (MS-01 ... MS-30).
MERGE / upsert	PostgreSQL MERGE: insert new keys, update existing keys when they changed.
PDM	Physical Data Model (deliverable 04).
PK	Primary Key: the column(s) that uniquely identify a row.
SCD 1 / SCD 2	Slowly changing dimension: Type 1 overwrites attributes, Type 2 keeps a new row per change.
SMD	This document: the narrative behind the SMX (approach, conventions, key rules).
SMX	Source-to-target mapping workbook: one row per target column (05_SMX_Source_to_Target_Mapping.xlsx).
SQL	Structured Query Language.

Term	Definition
Surrogate key	Integer key generated in the warehouse (SERIAL) and used by facts (e.g. company_key).
Version	One extracted copy of a source record. Staging holds the latest batch; intermediate keeps the latest version.
Watermark	Last extracted updated_at per source table, stored in metadata.pipeline_watermarks.
YYYYMMDD	Date written as year, month, day (e.g. 20260930); the format of date_key.

2. Mapping Scope & Layers

Table 3: Mapping Scope & Layers

From Layer	To Layer	Objects	Scripts
Source (SQL Server b2b_source_db + 2 CSV files)	Staging	8 tables + marketing_leads.csv, web_logs.csv -> 10 staging tables	python/pipelines/sql_server_pipeline.py, marketing_leads_pipeline.py, web_logs_pipeline.py
Staging	Intermediate	10 tables (clean, validated, latest version) + rejected_records	postgresql/intermediate/transform/Transform_*.sql (10)
Intermediate	Warehouse	6 dimensions (incl. generated dim_date) + 4 partitioned facts	postgresql/warehouse/Load/load_*.sql (10), PARTITIONS_SETUP/partitioning.sql
Warehouse	Marts	15 KPI views	postgresql/marts/KPI1-15.sql (mapped in the KPI Definition Document, 09)

3. Mapping Conventions

Table 4: Mapping Conventions

Code	Transformation Type	Meaning
DM	Direct move	Value copied unchanged (name and type may be identical or only renamed). (185 mappings)
TR	Trim / cleanse	Text cleansed: TRIM, blank-to-NULL, case standardisation (LOWER e-mail, UPPER http_method). (49 mappings)
CN	Convert data type	Data type converted (SQL Server/CSV type -> PostgreSQL type) during extract/load. (58 mappings)
DV	Derived / calculated	Value calculated from other columns or generated (date attributes, item_count, conversion_status). (13 mappings)
LK	Lookup	Value obtained by joining to another table (dimension surrogate keys, category_name, order statuses). (17 mappings)
SK	Surrogate key	Integer key generated by the warehouse (SERIAL). (5 mappings)
S2	SCD Type 2	SCD Type 2 control column (effective_start_date, effective_end_date, is_current). (3 mappings)
SY	System / audit column	System / audit column populated by the platform (_loaded_at). (20 mappings)

4. Object-Level Mapping Summary

Table 5: Object-Level Mapping Summary

Source Object	Staging	Interm.	Warehouse	Load	Script(s)
source. Companies	companies	companies	dim_ companies (SCD2)	CDC; SCD2	Transform_ companies / load_dim_ companies
source. Categories	categories	categories	dim_ products. category_ name (flattened)	CDC; flattened	Transform_ categories / load_dim_ products
source. Customers	customers	customers	dim_ customers (SCD1)	CDC; SCD1	Transform_ customers / load_dim_ customers
source. Suppliers	suppliers	suppliers	dim_suppliers (SCD1)	CDC; SCD1	Transform_ suppliers / load_dim_ suppliers
source. Products	products	products	dim_products (SCD1)	CDC; SCD1	Transform_ products / load_dim_ products
source. Supplier_ Product_ Mapping	supplier_ product_ mapping	supplier_product_mapping	dim_supplier_ product (SCD1)	CDC; SCD1	Transform_ supplier_ product_ mapping / load_dim_ supplier_ product
source. Orders	orders	orders	fact_orders; also fact_ order_items (status, lead), fact_leads. order_id	CDC; fact upsert	Transform_ orders / load_ fact_orders
source. Order_Items	order_items	order_items	fact_order_ items; fact_ orders.item_ count	CDC; fact upsert	Transform_ order_items / load_fact_ order_items
marketing_ leads.csv	marketing_ leads	marketing_leads	fact_leads	Snapshot; fact upsert	Transform_ marketing_ leads / load_ fact_leads
web_logs. csv	web_logs	web_logs	fact_web_ logs	Snapshot; insert-only	Transform_ web_logs / load_fact_ web_logs
(generated calendar)	-	-	dim_date	Generated	load_dim_ date

5. Key Transformation Rules

Table 6: Key Transformation Rules

Rule ID	Description	Applies To	Implemented In
R01	Incremental extract: only rows with updated_at > last watermark	8 SQL Server tables	sql_server_pipeline.py
R02	CSV snapshot: TRUNCATE staging table and re-append the full file (one transaction)	marketing_leads, web_logs	*_pipeline.py (CSV)
R04	Keep latest version per business key (ROW_NUMBER)	All 10 intermediate tables	Transform_*.sql
R05-R07	TRIM + blank-to-NULL; LOWER e-mail; UPPER http_method	Text columns	Transform_*.sql
R08-R12	Mandatory, domain, range, non-negative and timestamp-order checks	All intermediate tables	Transform_*.sql (validated_* CTE)
R13-R17	Parent must exist; buyer/supplier role; catalog >= cost; line total = qty x price - discount; order consistent with customer	Child tables	Transform_*.sql
R18	MERGE upsert, update only when source.updated_at is newer (idempotent)	Intermediate	Transform_*.sql
R19	Lead conversion: 'Not Converted' on insert, 'Converted' when referenced by an order	marketing_leads	Transform_marketing_leads.sql, Transform_orders.sql
R20	SCD Type 2 on 7 tracked company attributes	dim_companies	load_dim_companies.sql
R21	SCD Type 1 overwrite with NULL-safe change detection (IS DISTINCT FROM)	Other dimensions	load_dim_*.sql
R23-R24	Surrogate key lookup; point-in-time company version at order date	Facts	load_fact_*.sql
R25, R30	date_key = YYYYMMDD; facts routed to monthly partitions	Facts	load_fact_*.sql, partitioning.sql
R27-R28	item_count per order; order attached to originating lead	fact_orders, fact_leads	load_fact_orders.sql, load_fact_leads.sql
R33	Invalid CSV rows saved with a reason in intermediate.rejected_records (once per record)	marketing_leads, web_logs	Transform_marketing_leads.sql, Transform_web_logs.sql
R34	Staging emptied after a successful transform, once every row is in intermediate or rejected_records	All 10 staging tables	run_pipeline.py (clear_staging)

6. Deduplication & Latest-Record Logic

Each SQL Server extract appends new versions to staging, and staging is emptied after each successful run. It can still hold several rows per business key (e.g. after a failed run), so the transform must pick one.

Every transform starts with a `latest_*` CTE that ranks the versions of each business key and keeps rank 1:

```
ROW_NUMBER() OVER (
  PARTITION BY <business_key>
  ORDER BY _loaded_at DESC, updated_at DESC
) AS rn      -- ... WHERE rn = 1
```

- **_loaded_at DESC first:** the most recently landed copy wins, even if two versions carry the same updated_at.
- **updated_at DESC second:** tie-breaker when several versions arrive in the same load.
- **web_logs** has no updated_at and orders by _loaded_at only (the table is truncated each run, so versions do not accumulate).
- Rows with a NULL business key are removed in the same step (`WHERE <business_key> IS NOT NULL`).

The MERGE that follows is idempotent: an existing row is only updated when the incoming updated_at is newer (`WHEN MATCHED AND target.updated_at < source.updated_at`), so re-running a transform on the same staging data changes nothing. Before staging is emptied, a check confirms every staging row is in intermediate or in rejected_records (section 9).

7. Cleansing & Standardisation

Cleansing happens in the `cleaned_*` CTE of each transform, before validation, so that validation sees standardised values:

- **Trim and blank-to-NULL:** `NULLIF(TRIM(col), '')` on text columns. A value of spaces only becomes NULL, and the NOT NULL rules then treat it as missing.
- **Case standardisation:** e-mails are lower-cased so that the unique e-mail constraint is case-insensitive in practice; HTTP methods are upper-cased before the domain check.
- **Identifier trimming:** foreign-key IDs arriving as text (orders.customer_id / company_id / lead_id, order_items IDs) are trimmed so joins to parent tables do not fail on padding.
- **Domain (enum) validation:** company_type, gender, order_status, payment_status, funnel_stage and http_method must be in their allowed lists (SMX sheet Code_Values). Values are case-sensitive as generated: order_status is title case, payment_status lower case.
- **Range and consistency checks:** rating 1.0-5.0, lead_score 1-100, status_code 100-599, non-negative amounts, catalog_price >= cost_price, line_total = ROUND(quantity x unit_price - discount_amount, 2), updated_at >= created_at.
- **Type conversion:** no explicit CAST in the extract queries; pandas and SQLAlchemy convert NVARCHAR->VARCHAR, DATETIME2->TIMESTAMP, BIT->BOOLEAN and DECIMAL->NUMERIC while loading staging. CSV dates are parsed with `parse_dates`.

8. Key Handling

- **Business keys** (company_id, customer_id, order_id, ...) are 32-character hex UUIDs generated by the Python data generator. They are carried unchanged through staging and intermediate (PK there) and kept in the warehouse as the natural key of every dimension and as degenerate keys on facts (order_id, order_item_id, lead_id, log_id).
- **Surrogate keys** (company_key, customer_key, supplier_key, product_key, supplier_product_key) are SERIAL integers generated in the warehouse. Facts reference dimensions only through these keys.
- **Key lookup:** fact loads join each dimension on its business key to obtain the surrogate key. The joins are INNER joins, so a fact row whose dimension row is missing is not loaded (prevented by loading dimensions first).

- **Point-in-time lookup for SCD2:** company_key is the version of the company valid when the order was placed:

```
o.order_date >= com.effective_start_date
AND (com.effective_end_date IS NULL OR o.order_date < com.effective_end_date)
```

- **Date key:** integer YYYYMMDD from dim_date, joined on the event date (order_date, log_timestamp, lead created_at). It is also the partition key of all four facts. dim_date covers 2 years back to 6 months ahead (912 days currently).
- **Composite fact keys:** partitioned facts have primary keys (business key, date_key) because PostgreSQL requires the partition column in the primary key. The fact MERGE statements match on both columns.
- **Business-key references kept as attributes:** fact_orders.lead_id and fact_leads.order_id link orders and leads without a dimension (lead_id/order_id are degenerate).

9. Rejects & Error Handling

Validation is implemented as a filter (validated_* CTE): rows that fail a rule are not merged into intermediate. For the two CSV sources, a step before the MERGE saves them in intermediate.rejected_records with a reason (R33). SQL Server rows are protected by the source's own constraints.

How it is checked: before staging is emptied (R34), run_pipeline.clear_staging() counts staging rows that are neither in intermediate nor in rejected_records. If any exist, staging is kept and the run fails:

```
SELECT COUNT(*) FROM staging.orders s
WHERE NOT EXISTS (SELECT 1 FROM intermediate.orders i WHERE i.order_id = s.order_id)
AND NOT EXISTS (SELECT 1 FROM intermediate.rejected_records r
                WHERE r.table_name = 'orders' AND r.record_id = s.order_id);
```

Result on the current data set: 0 unaccounted rows. 2,258 leads and 4,517 web logs are in rejected_records (about 1.5% of the CSV rows, broken on purpose by the generator).

Pipeline-level errors: the intermediate transform executes all 10 scripts in one transaction, so a failing script rolls back the whole stage. The run is recorded in metadata.pipeline_run_log with status 'failed' and the error message, and watermarks are only advanced after a successful load, so the next run re-extracts the same rows.

Warehouse: unresolved lookups (missing dimension row or date outside dim_date) drop the fact row through the INNER JOIN. The warehouse unit tests (fact_*_tests.sql) and the layer reconciliation test detect this: intermediate and warehouse counts currently match for all facts.

Figure 1 shows the rejected rows per table and reason, as stored in `intermediate.rejected_records`.

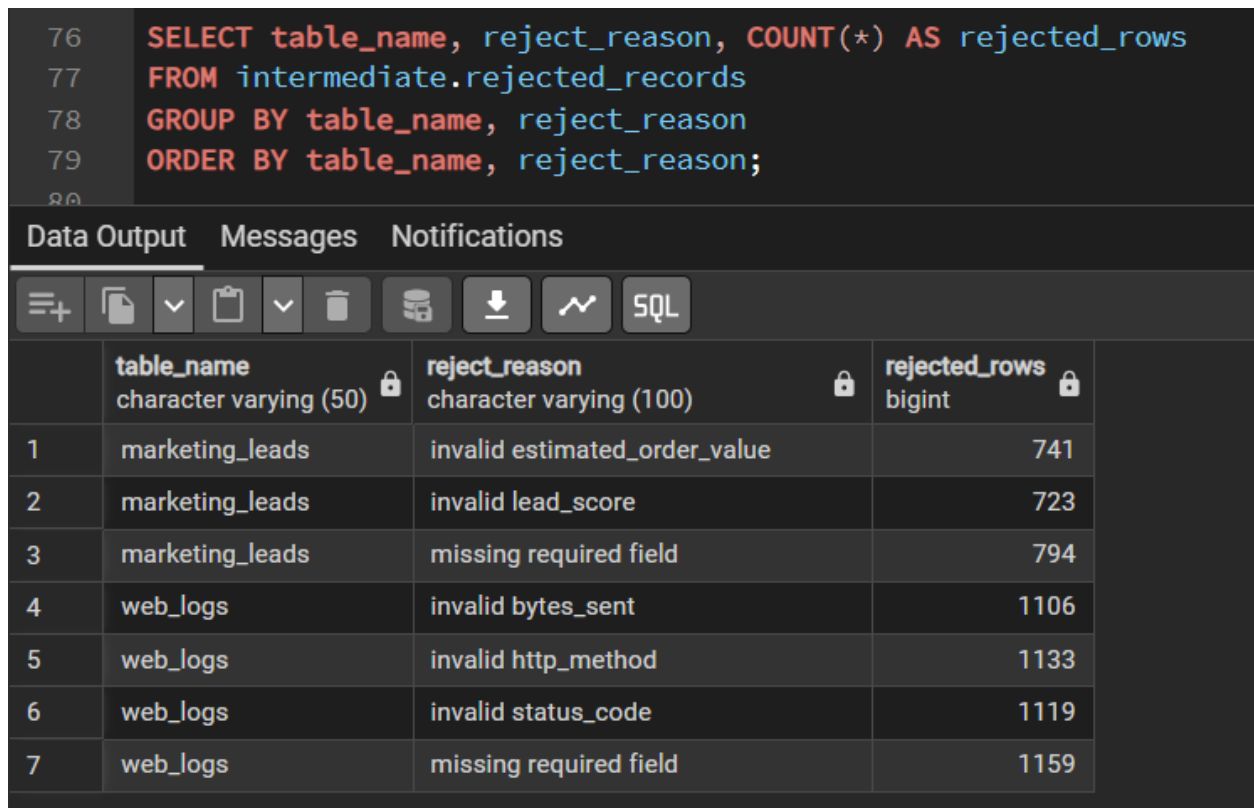


Figure 1: Rejected rows by table and reason (`intermediate.rejected_records`)

10. Reference to SMX Workbook

Column-level mappings: 05_SMX_Source_to_Target_Mapping.xlsx (sheet 'Column_Mapping').

11. Assumptions, Constraints & Open Issues

Table 7: Assumptions, Constraints & Open Issues

ID	Type	Description	Owner	Status
A1	Assumption	Source updated_at is set on every insert and update; the watermark CDC relies on it (BRD: CDC efficiency).	Data Engineering	Accepted
A2	Assumption	The two CSV files always hold the complete data set, so they are reloaded as snapshots.	Data Engineering	Accepted
C1	Constraint	Physical deletes are not captured by watermark CDC; the source never deletes (products use is_active).	Data Engineering	Accepted